

UDOM FYP PROJECT 2025 FAKE NEWS DETECTION PROJECT

Dataset contains the Following Columns :

- job id - ID number provided to each and every Job provided in the Dataset.
- title - Job Title provided.
- location - Job Location / Posting as per the Dataset.
- salary_range - Range for Salary by the Employer.
- company_profile - Short Description about the Company.
- description - Job Description provided on the Job Opportunity.
- requirements - Job Requirements to be fulfilled for the Job.
- benefits - Employee Benefits provided by the company.
- telecommuting - Whether the Job requires telecommuting works.
- has_company_logo - Whether the Company has its own Company Logo or not.
- has_questions - Whether the Company has asked questions or not.
- employment_type - Category of Employment provided for the Employees.
- required_experience - Type of the Previous Experience required from the Employees.
- required_education - Education Qualifications required from the Employees.
- Industry - Domain in which the Company does work into.
- Function - Type of Profession the Company follows.
- Fraudulent - Whether the Job Posting is Real or Fake

```
from nltk.stem.porter import PorterStemmer

ps = PorterStemmer()

corpus = []

for i in range(0, len(message)):

    review = re.sub('[^a-zA-Z]', ' ', message['description'][i])

    review = review.lower()
```

```
review = review.split()
```

```
review = [ps.stem(word) for word in review if not word in stopwords.words('english')]
```

```
review = ' '.join(review)
```

```
corpus.append(review)
```

Sure! Let's break down the Python code you shared **line by line**, explaining what each part does — especially in the context of **text preprocessing** using nltk and regular expressions.

Full Code:

```
from nltk.stem.porter import PorterStemmer
```

```
ps = PorterStemmer()
```

```
corpus = []
```

```
for i in range(0, len(message)):
```

```
    review = re.sub('[^a-zA-Z]', ' ', message['description'][i])
```

```
    review = review.lower()
```

```
    review = review.split()
```

```
    review = [ps.stem(word) for word in review if not word in stopwords.words('english')]
```

```
    review = ' '.join(review)
```

```
    corpus.append(review)
```

Line-by-line Explanation:

 **from nltk.stem.porter import PorterStemmer**

- Imports the **PorterStemmer** class from nltk (Natural Language Toolkit).
- PorterStemmer is used to **reduce words to their stem/root form**.

- Example: running, runs, ran → run
-

`ps = PorterStemmer()`

- Creates an object `ps` of the `PorterStemmer` class.
 - This object will be used to call the `.stem()` method on words.
-

`corpus = []`

- Initializes an empty list called `corpus`.
 - This will store the cleaned and processed text (each item in the list is one cleaned message).
-

`for i in range(0, len(message)):`

- Loops through each record (row) in the message list or DataFrame.
 - Assumes message is a **pandas DataFrame** with a column description.
-

`review = re.sub('[^a-zA-Z]', ' ', message['description'][i])`

- Uses **regex** to remove all characters except alphabets (a-zA-Z) from the text.
 - Replaces them with a space ' '.
 - Purpose: clean out numbers, punctuation, special symbols, etc.
-

`review = review.lower()`

- Converts all text to **lowercase**.
 - Standard practice to avoid treating "Apple" and "apple" as different words.
-

`review = review.split()`

- Splits the string into a **list of words**.

- Default split happens at whitespace.
-

 `review = [ps.stem(word) for word in review if not word in stopwords.words('english')]`

- This is a **list comprehension**:
 - Removes **stopwords** (like "the", "is", "and", etc.)
 - Stems each remaining word using the PorterStemmer.
- Result: shorter, meaningful, stemmed words.

Example:

Input: "This is running faster than expected"

Output after this step: ["run", "faster", "expect"]

 `review = ' '.join(review)`

- Joins the list of processed words back into a **single string**.
 - Spaces are added between words.
-

 `corpus.append(review)`

- Adds the cleaned and processed string to the corpus list.
-

 **Final Output:**

- corpus will be a list of preprocessed text strings.
- Each string is from the description field of your input and is:
 - Lowercased
 - Cleaned (no punctuation/numbers)
 - Tokenized
 - Stopwords removed
 - Stemmed

- Re-joined

✅ Example

Input:

```
message['description'][i] = "The cars are running faster than they were last week!"
```

After processing:

```
"car run faster last week"
```

Let me know if you want to extend this to include lemmatization, vectorization (TF-IDF or CountVectorizer), or saving the corpus for model training!

Let's go through the three lines of code step-by-step, explaining each in detail. This code is typically used in **text preprocessing** for training NLP models (e.g. LSTM, CNNs) in Keras.

💡 Code:

```
sent_length = 40
```

```
embedded_docs = pad_sequences(onehot_repr, padding='pre', maxlen=sent_length)
```

```
print(embedded_docs)
```

🧠 1. `sent_length = 40`

- **What it does:** Sets a fixed length for input sequences.
- **Why:** Neural networks like LSTMs need inputs of the **same length**. So, all text inputs will be **padded or truncated** to 40 tokens.
- You can change 40 based on the average or max sentence length in your dataset.

🧠 2. `embedded_docs = pad_sequences(onehot_repr, padding='pre', maxlen=sent_length)`

Function: pad_sequences (from tensorflow.keras.preprocessing.sequence)

- **Input:** onehot_repr — a list of sequences of integers, typically from one-hot encoding or tokenization.

Example:

- onehot_repr = [[1, 45, 23], [13, 5, 67, 8, 23]]
 - **Parameters:**
 - padding='pre': Adds zeros **at the beginning** of the sequence (e.g., [0, 0, 0, 1, 45, 23]).
 - maxlen=sent_length: Sets the final length of all sequences to 40.
 - If a sequence is **shorter**, it's padded.
 - If it's **longer**, it's truncated.
 - **Output:** embedded_docs — a NumPy array with all sequences of exactly 40 elements.
-

Example:

If onehot_repr = [[4, 55, 22], [15, 6]]

And sent_length = 5

Then pad_sequences(...) with padding='pre' will produce:

```
[[ 0  0  4 55 22]
```

```
[ 0  0  0 15  6]]
```

3. print(embedded_docs)

- Prints the padded sequences.
 - Useful for verifying that padding/truncation worked correctly and the shapes match expected input for the model.
-

Summary:

Line	Purpose
<code>sent_length = 40</code>	Set max input length
<code>pad_sequences(...)</code>	Pad/truncate input sequences to the same length
<code>print(...)</code>	Show the result for debugging or verification

Let me know if you want to see how this fits into a full model pipeline (embedding layer, LSTM/CNN, etc.).

```

embedding_vector_features = 50

model1 = Sequential()

model1.add(Embedding(voc_size, embedding_vector_features, input_length=sent_length))

model1.add(Bidirectional(LSTM(100)))

model1.add(Dropout(0.3))

model1.add(Dense(1, activation='sigmoid'))

# Compile

model1.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])

# Build model with input shape

model1.build(input_shape=(None, sent_length))

# Summary

print(model1.summary())

```

Let's walk through the entire block **line by line**, explaining the meaning and purpose of each line in this Keras deep learning model using **LSTM for binary classification**:

Line-by-Line Breakdown:

`embedding_vector_features = 50`

- Sets the size of the word embedding vector.
 - Each word will be represented by a **dense vector of size 50**.
 - Think of this as the number of features used to describe each word.
-

`model1 = Sequential()`

- Creates a **Sequential model** in Keras.
 - This means layers are stacked **one after another** (i.e., linear stack of layers).
-

`model1.add(Embedding(voc_size, embedding_vector_features, input_length=sent_length))`

- Adds an **Embedding Layer**:
 - `voc_size`: Size of the vocabulary (total unique words).
 - `embedding_vector_features`: Output dimension (e.g., 50) for each word vector.
 - `input_length=sent_length`: Length of each input sequence (e.g., 40 from earlier).

 Converts integer-encoded input (like [2, 45, 12, ...]) to dense 50-dimensional vectors.

`model1.add(Bidirectional(LSTM(100)))`

- Adds a **Bidirectional LSTM** layer with 100 units.
- Bidirectional means it reads the input **forward and backward**, helping to capture both past and future context in sequences.

 Output will be a 200-dimensional vector (100 forward + 100 backward).

`model1.add(Dropout(0.3))`

- Adds **Dropout regularization**.
 - Randomly sets 30% of the inputs to 0 during training to prevent **overfitting**.
-

`model1.add(Dense(1, activation='sigmoid'))`

- Adds a **Dense (fully connected) output layer** with:
 - 1 neuron
 - sigmoid activation → outputs a probability between 0 and 1

 Perfect for **binary classification** (e.g., spam vs. not spam, positive vs. negative sentiment).

`model1.compile(loss='binary_crossentropy', optimizer='adam', metrics=['accuracy'])`

- **Compiles** the model:
 - `loss='binary_crossentropy'`: Used for binary classification problems.
 - `optimizer='adam'`: Adaptive optimizer that works well in most scenarios.
 - `metrics=['accuracy']`: Model will report accuracy during training.
-

`model1.build(input_shape=(None, sent_length))`

- Manually builds the model with a specified input shape:
 - `None` → batch size is flexible
 - `sent_length` → each input has exactly `sent_length` tokens (e.g., 40)

This step is sometimes optional, but helps define the model explicitly before training or calling `.summary()`.

`print(model1.summary())`

- Prints a **summary of the model** architecture:
 - Layer names
 - Output shapes of each layer
 - Number of trainable parameters
-
-

Here's a breakdown of your model summary and what each component means:

✓ Model Summary Explained (Line by Line)

1. embedding_3 (Embedding)

- **Output Shape:** (None, 40, 50)
 - None: Batch size is flexible.
 - 40: Sentence length (each input is padded to 40 tokens).
 - 50: Each token is mapped to a 50-dimensional vector.
 - **Param #:** 250,000
 - Formula: $\text{vocab_size} * \text{embedding_dim}$
 - E.g., if $\text{vocab_size} = 5000$ and $\text{embedding_dim} = 50$,
 $5000 * 50 = 250,000$
- ◆ This layer converts each word index to a learned vector of 50 features.
-

2. bidirectional_3 (Bidirectional)

- **Output Shape:** (None, 200)
 - The LSTM inside has 100 units.
 - Bidirectional LSTM = 100 forward + 100 backward = **200 units output**.
- **Param #:** 120,800

- Formula for LSTM parameters:
 $4 * (\text{units} * (\text{units} + \text{input_dim} + 1))$

In this case:

- $\text{units} = 100$
- $\text{input_dim} = 50$
 $4 * (100 * (100 + 50 + 1)) = 60,400$
Since it's **bidirectional**, total params = $60,400 * 2 = 120,800$

- ◆ This layer learns patterns in the sequence from both directions (past and future).
-

3. dropout_3 (Dropout)

- **Output Shape:** (None, 200)
- **Param #:** 0
 - Dropout has no parameters.
- Randomly sets 30% of inputs to zero during training to help prevent overfitting.

- ◆ This layer is **only active during training**, not during evaluation/inference.
-

4. dense_3 (Dense)

- **Output Shape:** (None, 1)
 - Single neuron output (for **binary classification**).
- **Param #:** 201
 - Formula: $\text{input_units} * \text{output_units} + \text{bias}$
 - $200 * 1 + 1 = 201$

- ◆ This outputs a probability between 0 and 1 using the **sigmoid activation**.
-

Total Parameters: 371,001

- All parameters are **trainable** (i.e., they will be updated during training).
-

Model Purpose:

This model is **designed for binary text classification**, e.g.:

- Spam vs. Not Spam
 - Positive vs. Negative Sentiment
 - Fake News Detection, etc.
-
-
-

```
from sklearn.model_selection import train_test_split
```

```
X_train, X_test, y_train, y_test = train_test_split(X_final, y_final, test_size=0.25,  
random_state=32)
```

```
model1.fit(X_train,y_train,validation_data=(X_test,y_test),epochs=12,batch_size=64)
```

Here's a clear explanation of each line in your code:

Line-by-Line Explanation

```
from sklearn.model_selection import train_test_split
```

- This **imports** the `train_test_split` function from `scikit-learn`, which is used to split your data into training and testing sets.
-

```
X_train, X_test, y_train, y_test = train_test_split(X_final, y_final, test_size=0.25,  
random_state=32)
```

- **X_final**: Your input features (e.g. padded text sequences).

- **y_final**: Your labels (e.g. 0 or 1 for binary classification).
 - **test_size=0.25**: 25% of the data will be used for testing, 75% for training.
 - **random_state=32**: Ensures the split is reproducible. Same random split every time you run the code.
-

model1.fit(X_train, y_train, validation_data=(X_test, y_test), epochs=12, batch_size=64)

- **model1.fit()**: Trains the model on the training data.
 - **X_train, y_train**: Training data used to train the model.
 - **validation_data=(X_test, y_test)**: During training, it also evaluates the model's performance on the test data after each epoch.
 - **epochs=12**: The model will go through the **entire training dataset 12 times**.
 - **batch_size=64**: During each epoch, training data is divided into **batches of 64 samples** to update weights.
-

What This Code Does Overall:

You're training your deep learning model on a portion of your dataset and evaluating its performance on unseen data (validation/test) to monitor how well it's generalizing.

Let me know if you'd like help plotting **accuracy/loss curves**, adding **early stopping**, or saving the **best model**.

You're using a **confusion matrix** to evaluate your binary classification model, which is a great way to understand the **true/false positives/negatives**. Let's break it all down:

Raw Confusion Matrix:

```
array([[3541, 52],  
       [ 93, 110]])
```

Meaning of the Matrix (Binary Classification):

	Predicted: 0	Predicted: 1
Actual: 0	3541 (True Negatives)	52 (False Positives)
Actual: 1	93 (False Negatives)	110 (True Positives)

Interpretation:

- True Negatives (TN = 3541): The model correctly predicted class 0.
 - False Positives (FP = 52): The model incorrectly predicted 1 instead of 0.
 - False Negatives (FN = 93): The model incorrectly predicted 0 instead of 1.
 - True Positives (TP = 110): The model correctly predicted class 1.
-

Metrics You Can Compute:

Accuracy:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{110 + 3541}{110 + 3541 + 52 + 93} \approx 96.5\%$$

Precision (Positive Predictive Value):

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{110}{110 + 52} \approx 67.9\%$$

Recall (Sensitivity / True Positive Rate):

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{110}{110 + 93} \approx 54.2\%$$

F1 Score (Harmonic mean of precision and recall):

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \approx 60.3\%$$



The Plot Code You Wrote:

You're generating a visual **heatmap** of the confusion matrix using seaborn. It helps quickly see where your model is making errors.

```
plt.figure(figsize=(6,4))

sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel('Predicted Labels')

plt.ylabel('True Labels')

plt.title('Confusion Matrix')

plt.show()
```

- `annot=True`: Displays the numbers inside the boxes.
 - `fmt='d'`: Format as integers.
 - `cmap='Blues'`: Blue-colored heatmap.
-



Next Steps (Optional but Helpful):

- Compute and display classification metrics using:

```
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred))
```

- Add class names to the heatmap:

```
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=['Negative', 'Positive'],
yticklabels=['Negative', 'Positive'])
```

Would you like me to help generate the precision, recall, and f1-score values using your data?

Let's go through each line of your code step by step to explain what it's doing. This code is for **preprocessing a job description + requirements** and **predicting if it's fake or real** using your trained model1.

Imports

```
from tensorflow.keras.preprocessing.text import one_hot
```

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

- **one_hot**: Converts words into integers (basic form of tokenization).
- **pad_sequences**: Ensures all sequences are the same length by adding padding (0s) either before or after.

Set Parameters

```
voc_size = 5000
```

```
maxlen = 40
```

- **voc_size**: Vocabulary size used in the training — number of distinct tokens allowed.
- **maxlen**: Maximum number of tokens allowed in the input after padding (used in training as well).

Define Sample Inputs

```
description = "We are looking for a skilled data analyst to join our team."
```

```
requirements = "Must have 2+ years of experience with Python and SQL."
```

You define a sample job description and its requirements.

Combine Text

```
test_text = description + " " + requirements
```

You merge both strings into one input text.

Clean the Text

```
import re
```



```
test_text_cleaned = re.sub('[^a-zA-Z]', ' ', test_text) # Remove non-letters
```

```
test_text_cleaned = test_text_cleaned.lower() # Convert to lowercase
```

- **Remove special characters/numbers** (like +, ., etc.).
 - **Lowercase** everything for normalization.
-

Text to Numeric Tokens

```
encoded = one_hot(test_text_cleaned, voc_size)
```

- Each word is converted into an integer between 1 and 5000 using a simple hashing trick (not from your original vocabulary list).
 - The output is a list of integers.
-

☐ Pad the Sequence

```
padded = pad_sequences([encoded], maxlen=maxlen, padding='post')
```

- Makes sure the input sequence has a length of 40 (same as what your model was trained on).
 - Adds zeros at the **end** (padding='post') if it's shorter than 40 tokens.
-

Make a Prediction

```
prediction = model1.predict(padded)
```

```
print("Prediction score:", prediction)
```

- Runs the padded input through your trained model.
 - Output is a probability between 0 and 1.
-

Interpret the Result

```
if prediction >= 0.5:
```

```
    print("🚨 This job posting is likely FAKE.")
```

else:

```
print("✅ This job posting is likely REAL.")
```

- If the predicted score is **0.5 or higher**, it's **fake**.
 - If lower, it's considered **real**.
-

✅ Example Output

If the model returns:

Prediction score: [[0.81]]

🔔 This job posting is likely FAKE.
